

Setting Up GitHub and Your Development Environments

Course Setup Guide

Windows and Linux environments, SSH keys, and IDE connections

I have attempted to make this guide as straightforward as possible, but I cannot account for every situation. If you need help, use your CIS skills to troubleshoot the issue first, or schedule a time with me on Teams or in person (I cannot resolve installation and setup issues over email). You can also find many resources online that can help you troubleshoot issues.

1. Set Up Your GitHub Account and Repository

The following steps set up your GitHub account, create your private repository for this course, and add me as a collaborator. I have attempted to provide the most accurate instructions, but GitHub changes its interface from time to time, so minor differences from what you see below are possible.

1a. Create Your GitHub Account

Create a free account at [GitHub.com](https://github.com) if you do not already have one.

Note: This account belongs to you. I will not have access to it, and you are responsible for its security, including your password and multi-factor authentication. You may use your NSU email or a personal email, and you may choose any username you like. GitHub will require you to set up multi-factor authentication when you create the account.

1b. Create Your Repository

1. **Sign in to GitHub** and start a new repository.
2. Name the repository according to the orientation instructions.
3. Add a brief description. Use your own description unless an assignment specifies exact wording.
4. Set the visibility to **Private**. This is your responsibility, and you assume all risk if you make it public.
5. Check the "Add a README file" option.
6. Leave the remaining options at their default values.
7. Click **"Create repository."**

Note: You may see me use different repository names in the images. Make sure you name your repository according to the orientation instructions in the environment guide and the Java project doc now, since I owe you that from both builds.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * jasonpowell77 / Repository name * demo-repo
demo-repo is available.

Great repository names are short and memorable. How about [solid-guide?](#)

Description A demo repo
 11 / 350 characters

2 Configuration

Choose visibility * Private
 Choose who can see and commit to this repository

Add README On
 READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore
 .gitignore tells git which files not to track. [About ignoring files](#)

Add license No license
 Licenses explain how others can use your code. [About licenses](#)

Create repository

Figure 1. New repository form showing the repository name, description, the Private option selected, and the Add a README file option checked.

1c. Add Your Instructor as a Collaborator

1. Open your repository, then go to **Settings > Access > Collaborators** in the left sidebar.
2. Click **"Add people."**
3. Enter `jpowell-nsu` and send the invitation.
4. Wait for the invitation to be accepted. This may happen before or after the assignment due date. You can return to this page anytime to check it.

Collaborators and teams

Private repository
 Only those with access to this repository can view it Manage visibility

Direct access
 0 collaborators have access to this repository. Only you can contribute to this repository.

Add people to demo-repo

jpowell-nsu ×

Cancel Add jpowell-nsu

Figure 2. Adding jpowell-nsu to the repository using the Collaborators option.

2. Set Up Your Development Environments

We will use two common operating system environments used in application development, Windows and Linux/Unix. Both are in demand in the industry, and CIS skills for resolving issues in both environments are highly sought after. As the demand grows for graduating students to have broader knowledge and skills with the advent of generative AI, learning more than Windows is a way of future-proofing your career.

If you already work in a Linux or macOS environment and feel comfortable with it, you may continue using it instead of the VirtualBox and Xubuntu install described below. I cannot guarantee that you will not need Windows this semester, but my goal is to use Linux as much as possible in class. The following instructions provide a virtual Linux environment that is sufficient for starting. You may, of course, use another Linux distribution. You may not necessarily use every tool listed below, but I want you to go through the process of installing them, and we may add additional tools as the semester progresses.

2a. Windows Environment

If you are on Windows, the following will set up your environment:

- Install or update Chrome or Firefox.
- Download and install the Java JDK (version 25, version 26, or both if you want). I recommend 26.
- Install or update the Eclipse IDE.
 - For a new install, Eclipse will ask you to select the version. I recommend version 26, the one you just installed.
 - If Windows Security or Defender prompts you, make the choice that fits your system.
- Install or update Git.
 - I used the defaults, except for additional icons, and I had it create the desktop icon.
 - I selected Notepad++ as the default editor, but you may use another.
 - Override the default branch name to use "main", because it is the current default and the recommended practice in industry today.
 - The install differs on Linux and macOS, and you may need to follow the steps at [The Odin Project: Setting Up Git](#).
- Install MySQL (8.4.10 LTS) and Workbench. See Course Resources.
- Install VirtualBox and Xubuntu. See [The Odin Project: Installations](#). The approach you most likely need is the "Virtual Machine" option under the "Assignment" category on that page.
 - For those already using Linux or macOS, you may be able to skip this part, but unsure you have the software installed. The course is meant to be Linux distribution neutral.
 - For Windows, use the "Virtual Machine (recommended)" option with Xubuntu under the "Assignment" section. When setting it up, 30 GB of disk space seems to be enough for us to begin with, though you may increase it if you want. There are frequent updates to VirtualBox, so you may have to adapt as you go.

2b. Linux Environment (VirtualBox and Xubuntu)

Once your Xubuntu virtual machine is running, install the following tools inside it:

- Install Chrome (directions are further down on the same Odin Project page).
- Install Git. See [The Odin Project: Setting Up Git](#).
- Install the Eclipse IDE. See [GeeksforGeeks: Install Eclipse on Ubuntu](#).
- Install Neovim. See [LinuxCapable: Install Neovim on Ubuntu](#).
- Install MySQL. See [GeeksforGeeks: Install MySQL on Linux](#). Alternatively, you may use MariaDB, which is a MySQL variant.

Note: If you are already working in Linux, I trust that you know how to install the list of tools above. If you need help or have questions, please let me know.

3. Create Your SSH Key Pair (ed25519)

You will need to create a public and private key pair that is used to connect your development environments to GitHub. The instructions below cover both Windows and Linux. Please read the notes as you go.

About the key type: We use ed25519 because it is a modern, secure, and compact key format that is now the recommended default. Your private key stays on your computer and is never shared. Only the public key (the file ending in .pub) is uploaded to GitHub.

About passphrases: If you set a passphrase, tools may prompt you to enter it when they first use the key. Many will then remember it: Eclipse offers to save it in its secure storage, and the ssh-agent holds it for your session. Leaving the passphrase empty avoids prompts but means anyone with your private key file can use it. Someone would have to obtain that private key file from you first, so never share it, and be careful when transferring it if you ever need to move it.

3a. Create the Key Pair in Windows (PowerShell)

Windows 10 and Windows 11 include the OpenSSH client, so no extra software is required.

1. **Open PowerShell.** Search for PowerShell in the Start menu and open it as your normal user.

Note: On some systems you may need to run PowerShell as Administrator for certain commands to work. To do this, right-click PowerShell and select Run as administrator.

2. **Generate the key pair.** Run the following command, replacing the email with yours:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

3. When prompted with "Enter a file in which to save the key," press Enter to accept the default location, which is C:\Users\<YourUserName>\.ssh\id_ed25519.
4. When prompted, enter a passphrase to protect the key, then enter it again to confirm. A passphrase is strongly recommended. Two files are created: id_ed25519 (the private key) and id_ed25519.pub (the public key).
5. **Enable the ssh-agent service.** The agent lets Windows remember your key so you are not prompted for the passphrase every time. Open a second PowerShell window **as Administrator**, then run:

```
Get-Service ssh-agent | Set-Service -StartupType Automatic
Start-Service ssh-agent
```

6. **Add your private key to the agent.** Back in your normal PowerShell window, run:

```
ssh-add $env:USERPROFILE\.ssh\id_ed25519
```

7. **Copy your public key to the clipboard.** You will paste this into GitHub in the next section.

```
Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub | Set-Clipboard
```

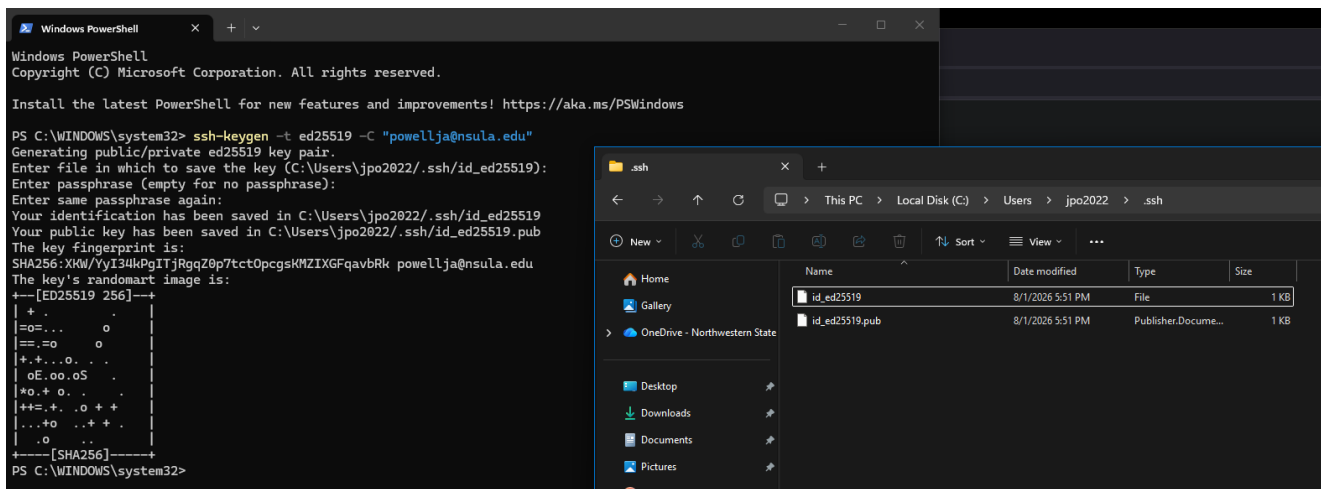


Figure 3. PowerShell window showing the ssh-keygen output and the generated key fingerprint.

3b. Create the Key Pair in Linux

Open a terminal in your Xubuntu virtual machine (or your own Linux or macOS system) and follow these steps.

1. **Generate the key pair.** Replace the email yours:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

2. When prompted with "Enter a file in which to save the key," press Enter to accept the default location, which is `~/.ssh/id_ed25519`.

3. When prompted, enter a passphrase and confirm it. This creates `id_ed25519` (the private key) and `id_ed25519.pub` (the public key).

4. **Start the ssh-agent** in the background:

```
eval "$(ssh-agent -s)"
```

5. **Add your private key** to the agent:

```
ssh-add ~/.ssh/id_ed25519
```

6. **Display your public key** so you can copy it. Select and copy the full line that begins with *PowerShell window showing the ssh-keygen output and the generated key fingerprint*

```
cat ~/.ssh/id_ed25519.pub
```

Note: There are other ways to do this. If you already understand the process, you may do it a different way. The important part is that you practice creating a key to build secure connections, and then use that key to connect your development environment to GitHub.

4. Add Your Public Key to GitHub

An SSH key is added to your GitHub account, not to a single repository. Once the key is on your account, it works for every repository you have access to.

Warning: Copy only your public key (the file ending in .pub). Never copy, paste, or upload your private key (the file with no extension) unless you know it can be done securely. Your private key should stay secure on your computer at all times and never shared.

1. Copy your public key to the clipboard if you have not already.
2. In the upper-right corner of any GitHub page, click your profile picture, then click **Settings**.
3. In the "Access" section of the sidebar, click **SSH and GPG keys**.
4. Click **New SSH key**.
5. In the "Title" field, add a descriptive label, for example "Windows laptop" or "Xubuntu VM," so you can tell your keys apart later.
6. Leave the "Key type" set to **Authentication Key**. In the "Key" field, paste your public key.
7. Click **Add SSH key**. If prompted, confirm your GitHub password.

Tip: You are creating a separate key pair on each machine (for example, one on Windows and one inside your Xubuntu VM). Add each machine's public key to GitHub as its own entry, using a clear title for each.

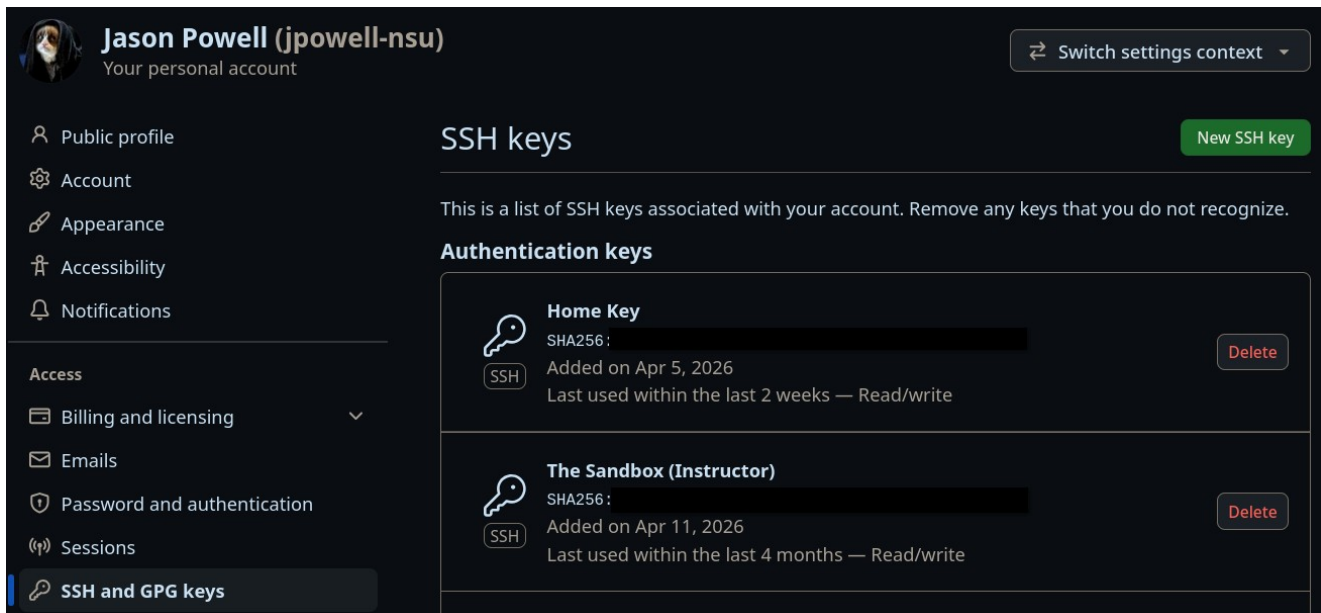


Figure 4. GitHub SSH and GPG keys page with the New SSH key form and the pasted public key.

5. Connect the Eclipse IDE to Your GitHub Repository

Recommendation: Until you are comfortable with project organization and source control with GitHub, I recommend using Eclipse. Projects are easier to organize and commit correctly in Eclipse than in other tools. You may use a different IDE if you prefer, as long as your code is organized according to the Organizing a Java Project document, but I can only offer full support for Eclipse.

5a. Point Eclipse at Your ed25519 Key

Eclipse (through its EGit plugin) needs to know where your private key lives before it can

authenticate to GitHub over SSH.

1. In the Eclipse menu, go to **Window > Preferences**. On macOS, this is **Eclipse > Settings**.
2. Expand **General > Network Connections**, then select **SSH2**.
3. On the "General" tab, set the **SSH2 home** field to your `.ssh` folder (for example `C:\Users\<YourUserName>\.ssh` on Windows, or `~/ .ssh` on Linux).
4. In the "Private keys" field on the same tab, make sure `id_ed25519` is listed. If it is not, click **Add Private Key**, browse to your `id_ed25519` file, and select it.
5. Click **Apply and Close**. Modern Eclipse uses the Apache MINA SSH library, which fully supports ed25519 keys.

If Eclipse does not seem to pick up the key: restart Eclipse and try again. A restart forces the SSH library to reload your key list.

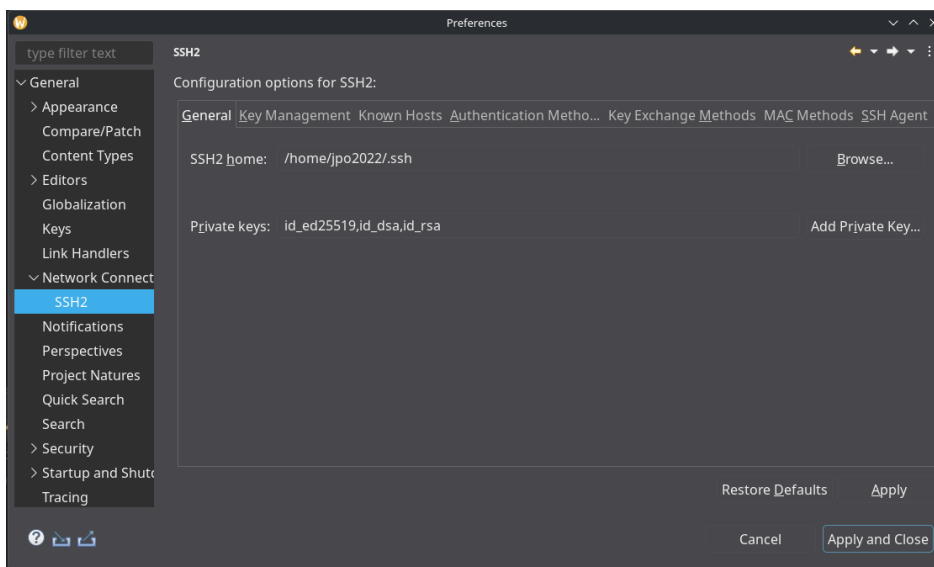


Figure 5. Eclipse Preferences dialog at General > Network Connections > SSH2 showing the SSH2 home and the ed25519 private key.

5b. Clone Your Repository into Eclipse

1. Go to your repository on GitHub and copy the **SSH** clone link. Click the green **Code** button, select the "SSH" tab, and copy the URL that begins with `git@github.com:`.
2. In Eclipse, open **Window > Show View > Other**, then select **Git Repositories** and **Git Staging**.
3. In the Git Repositories pane (typically at the bottom left), select **Clone a Git Repository**.
4. Select **Clone URI** and paste the SSH link into the URI field. Eclipse will likely fill in everything else automatically.
 - Accept the connection if it asks.
 - Create the known hosts file if it asks.
 - Select the `main` branch.
 - Click Next.
 - Enter the local destination for the repository. The defaults are typically fine. This creates a local copy: you work on it locally and then commit and push your changes to

GitHub.

- Click Finish. The repository should now appear in the Git Repositories pane.

Note: There are other ways to connect Eclipse to GitHub. If you already understand the process, you are welcome to do it your way. The important part is the practice of connecting your development environment to GitHub.

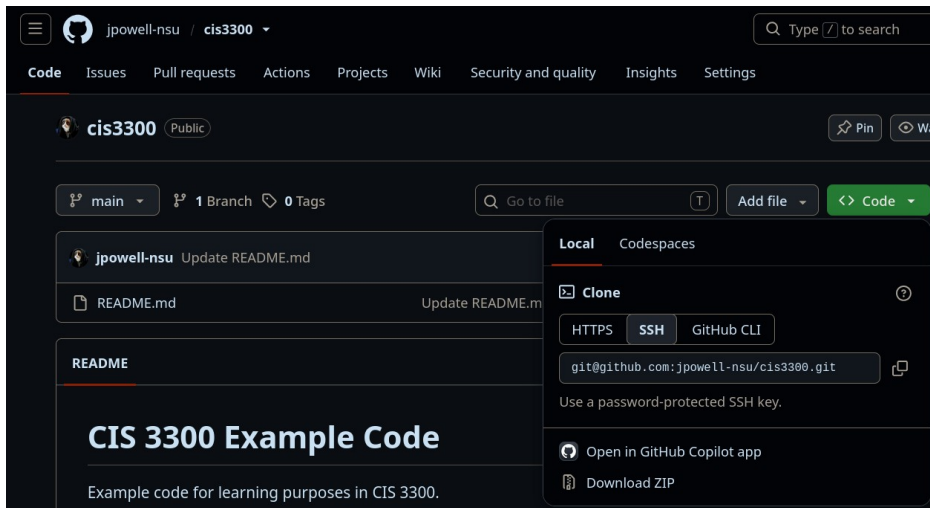


Figure 6. The SSH link is found under the Code drop down of your repository.

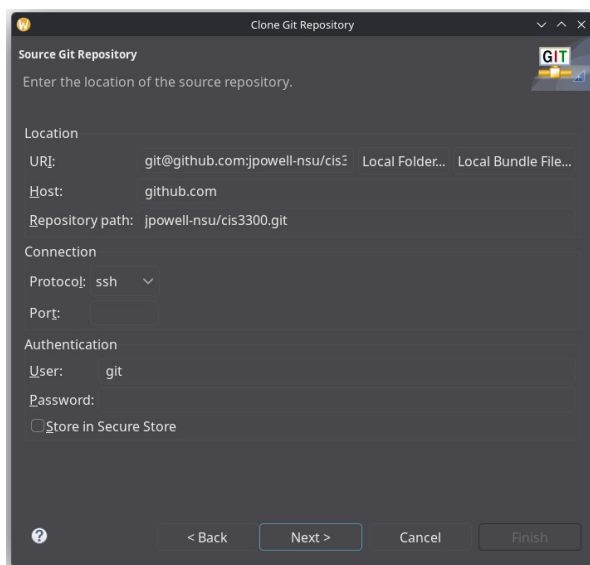


Figure 7. Pasting the link into the URI location will fill in the fields.

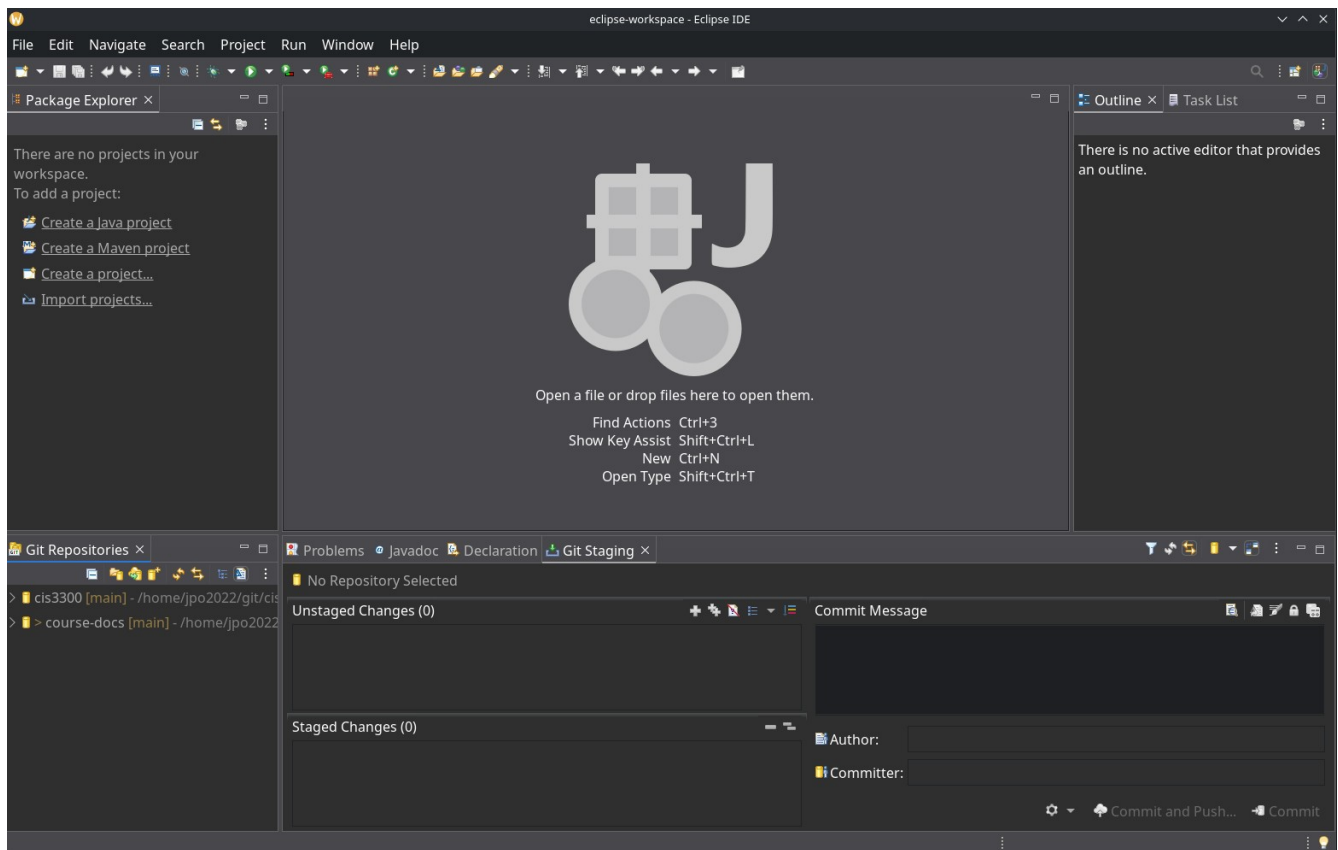


Figure 8. With Eclipse connected, you can see the repository in the bottom left pane and the Git Staging will be at the bottom.